

Proyecto de prácticas conjunto EI1039 – EI1048

Grado en Ingeniería Informática

proyecto

objetivo

aplicación de movilidad que permita…

- calcular rutas entre dos lugares de interés,
- elegir método de movilidad (p. ej., vehículo, bicicleta, a pie).
- conocer el trayecto, duración, tipo de ruta (p. ej., más rápida, más corta, etc.)
- obtener coste asociado según vehículo (p. ej., combustibles, kW, calorías para rutas en bicicleta/a pie)

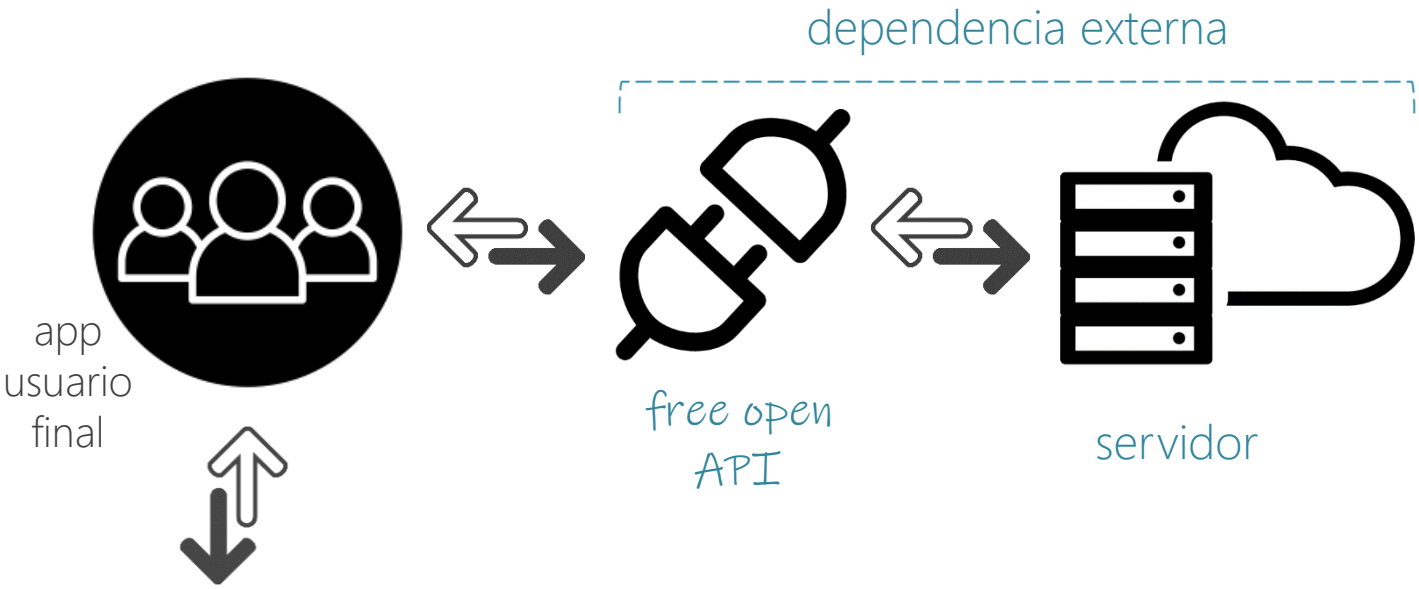


Fuente: https://www.freepik.es/

proyecto

resumen

proyecto de prácticas



- geocoding
- cálculo de rutas
- coste combustible
- precio €/kWh
- …

aplicación de movilidad que permita calcular rutas entre dos lugares de interés sobre mapas

proyecto

versiones

opciones

- versión básica → nota ≤ 8
  - requisitos funcionales y formativos especificados
  - tipos de aplicación: escritorio, web, móvil; en los primeros laboratorios se orientará sobre qué tecnología emplear en cada caso
- versión avanzada → nota ≤ 10 : versión básica + innovación en…
  - metodología (proceso): herramientas de gestión, planificación y seguimiento (Tablero Kanban, GitHub Projects), integración continua (Azure Pipelines, Github Actions), fichero README, etc.
  - tecnología (producto): servidores en la nube (Firebase, Google Cloud).
  - calidad del software: p. ej., diseñar imagen de marca (logotipo, paleta de colores, elementos tipográficos, etc.)

versión básica

alcance versión básica

requisitos funcionales

- gestión de usuarios
- gestión de lugares de interés a través de topónimos y/o coordenadas geográficas
- gestión de vehículos
- gestión de rutas
- gestión de favoritos
- persistencia de la información (p. ej., lugares de interés, vehículos, rutas, etc.)

requisitos formativos

- desarrollo según ATDD, incluyendo pruebas de aceptación e integración ejecutables
- seguir prácticas de desarrollo colaborativo (VCS, pair programming, etc.)
- abstracciones para API, formatos de intercambio de datos, servicios de datos, etc.
- arquitectura desacoplada: lógica independiente de GUI, BBDD y API
- código basado en (al menos 5) patrones de diseño (justificando necesidad)
- GUI con buena usabilidad

versión básica

historias de usuario

ejemplo de descomposición en historias de usuario

- Gestión de lugar de interés (topónimo, coord. geog.)
  - Como usuario quiero dar de alta una ubicación a partir de un topónimo para usarla en rutas
  - Como usuario quiero dar de alta una ubicación a partir de coordenadas geográficas para…
  - Como usuario quiero obtener las coord. geog. de una ubicación a partir de su topónimo…
  - Como usuario quiero obtener el topónimo más próximo a coord. geog. de una ubicación…
  - Como usuario quiero dar de baja una ubicación disponible…
- Gestión de rutas
  - Como usuario, dados dos lugares de interés y un método de movilidad, quiero obtener una ruta (trayecto, distancia y duración) entre ambos lugares para visualizarla.
  - Como usuario quiero conocer el coste asociado a una ruta en coche (coste de combustible) para facilitar la toma de decisiones.
  - Como usuario quiero conocer la ruta más rápida/corta/económica entre dos puntos para…
  - …

versión básica

requisitos formativos

EI1039 - Diseño de Software

- Se implementarán al menos cinco patrones de diseño, justificando su idoneidad y beneficios.
- Se implementará un patrón de diseño para las interfaces de usuario (MVC o similar), justificando además la usabilidad de la interfaz de usuario resultante.
- Se justificará elección de la arquitectura de software, identificando y justificando razonadamente las características más relevantes de la arquitectura.

versión básica

requisitos formativos

EI1048 - Paradigmas de Software

- El proyecto se desarrollará siguiendo la metodología ATDD.
- Se definirán interfaces (abstracciones) para modelar las interacciones entre la aplicación cliente y cada servicio público (API). Esto permitirá probar la aplicación con independencia del estado de los servicios (prueba de integración).
- Se definirá una interfaz para garantizar la independencia con respecto a cualquier solución de almacenamiento adoptada (para preservar estado de sesión).
- Se definirán interfaces para garantizar la independencia con respecto a los formatos de intercambio de datos y protocolos de comunicación usados por las API.
- Se diseñará una arquitectura desacoplada que garantice la independencia entre interfaz de usuario, lógica de negocio, base de datos y dependencias externas (API).

metodología de desarrollo

principio

"...el proceso será más importante que el producto final, aunque este último deberá cumplir todos los requisitos funcionales y formativos".

metodología de desarrollo

ATDD

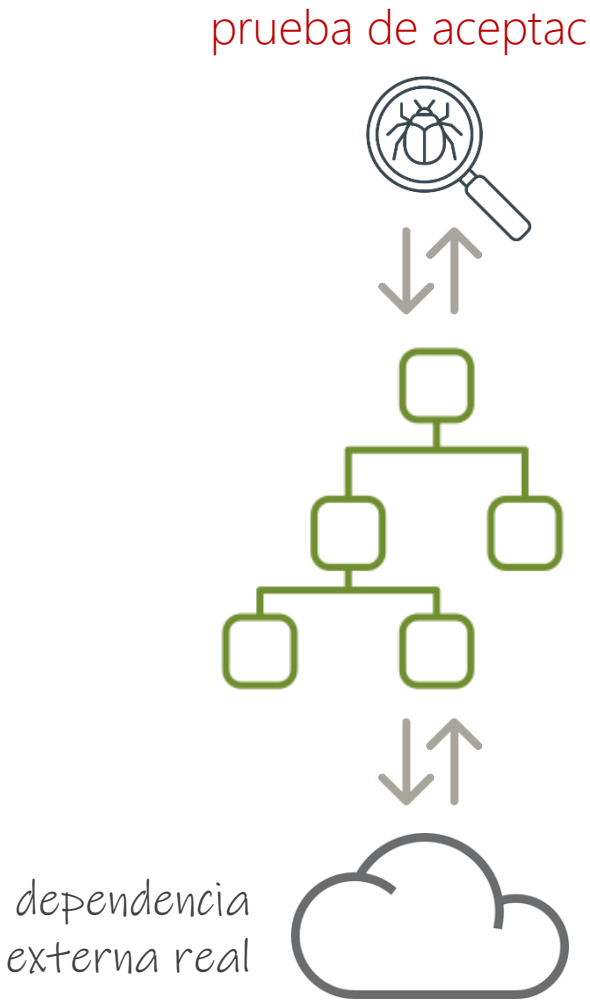
requisito →
user stories →
escenario(s) →
prueba(s) de aceptación →
end-to-end long skinny slices

ATDD

- Requisitos e historias de usuario, disponibles en versión básica.
- Dos escenarios relevantes por historias, uno válido y uno inválido.
- Análisis completo de cada escenario: alcance, casos extremos, etc.
- Una prueba de aceptación por cada escenario.
- Cada prueba respetará el formato Given – When – Then.

pruebas

aceptación

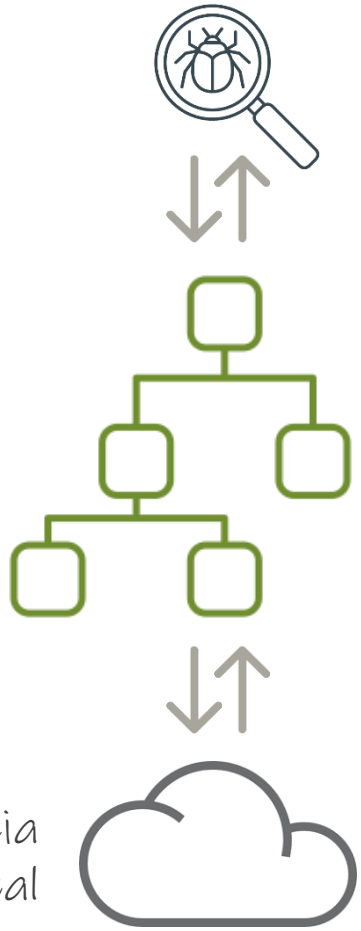


Directamente sobre la lógica de negocio de alto nivel, sin incluir la interfaz gráfica de usuarios.

pruebas

aceptación

prueba de aceptación

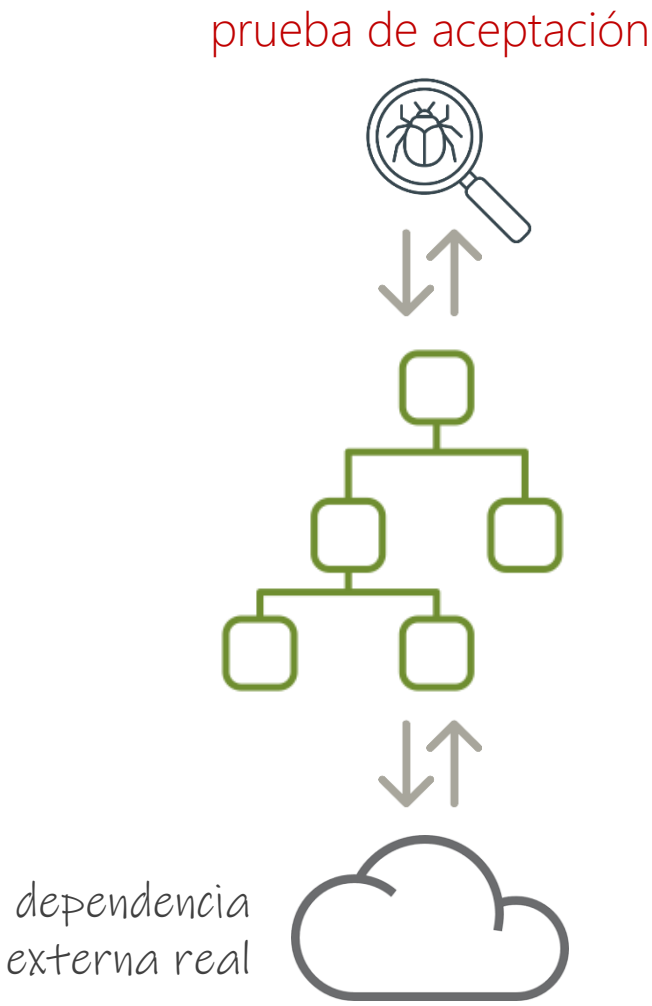


dependencia externa real

¿Y si no es predecible?

pruebas

aceptación



¿Y si no es predecible?

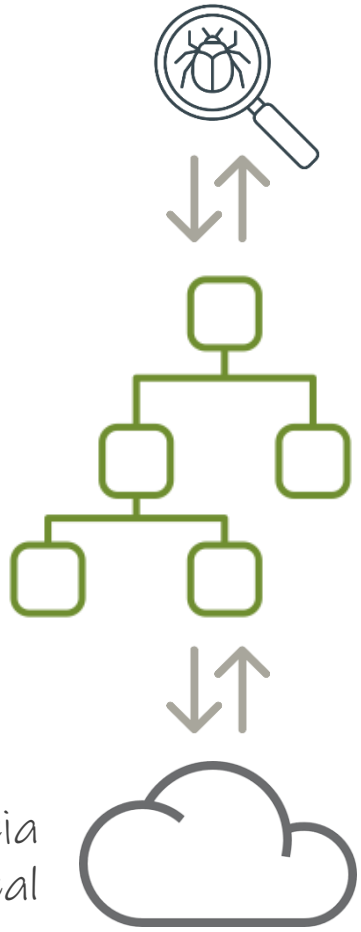
opciones:

- naive test: pasa si no ocurre excepción
- sanity test: comprobamos verosimilitud

pruebas

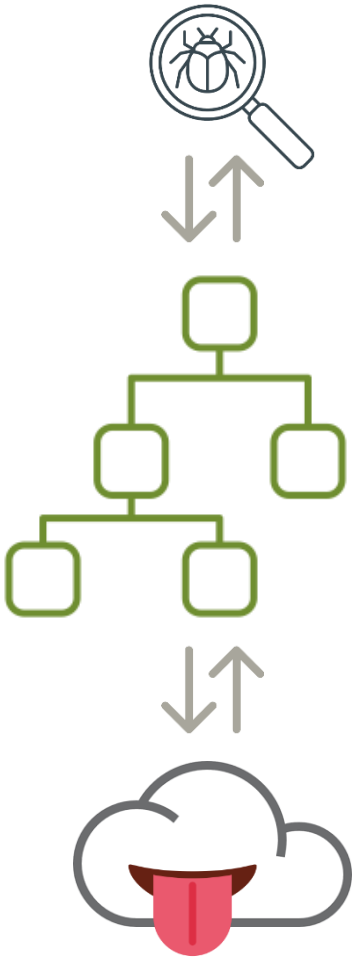
aceptación + integración

prueba de aceptación



dependencia externa real

prueba de integración

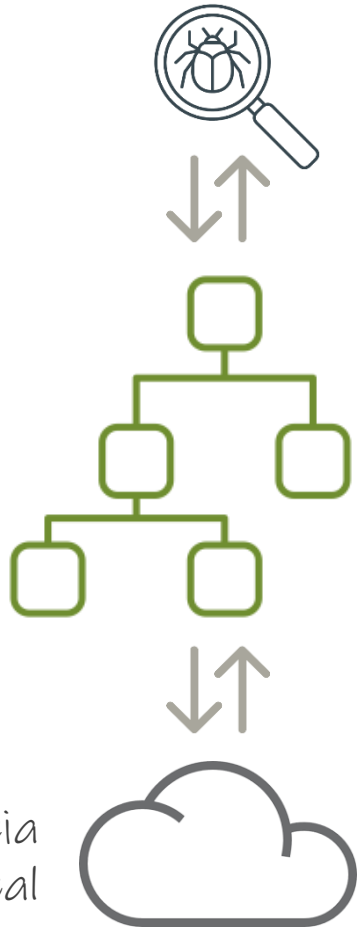


mock, fake

pruebas

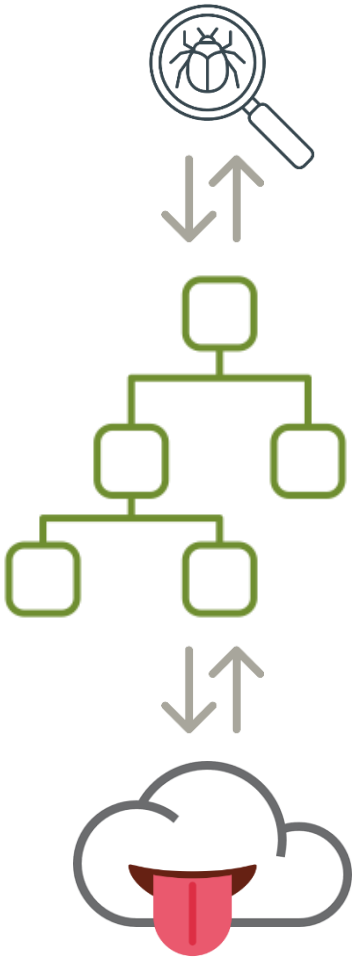
aceptación + integración

prueba de aceptación



dependencia externa real

prueba de integración



mock, fake

mock, fake...

- simula comportamiento de dependencia real
- configurable, ergo predecible
- sin lógica
- disponible
- ejecución rápida

architecture spike

iteración 0

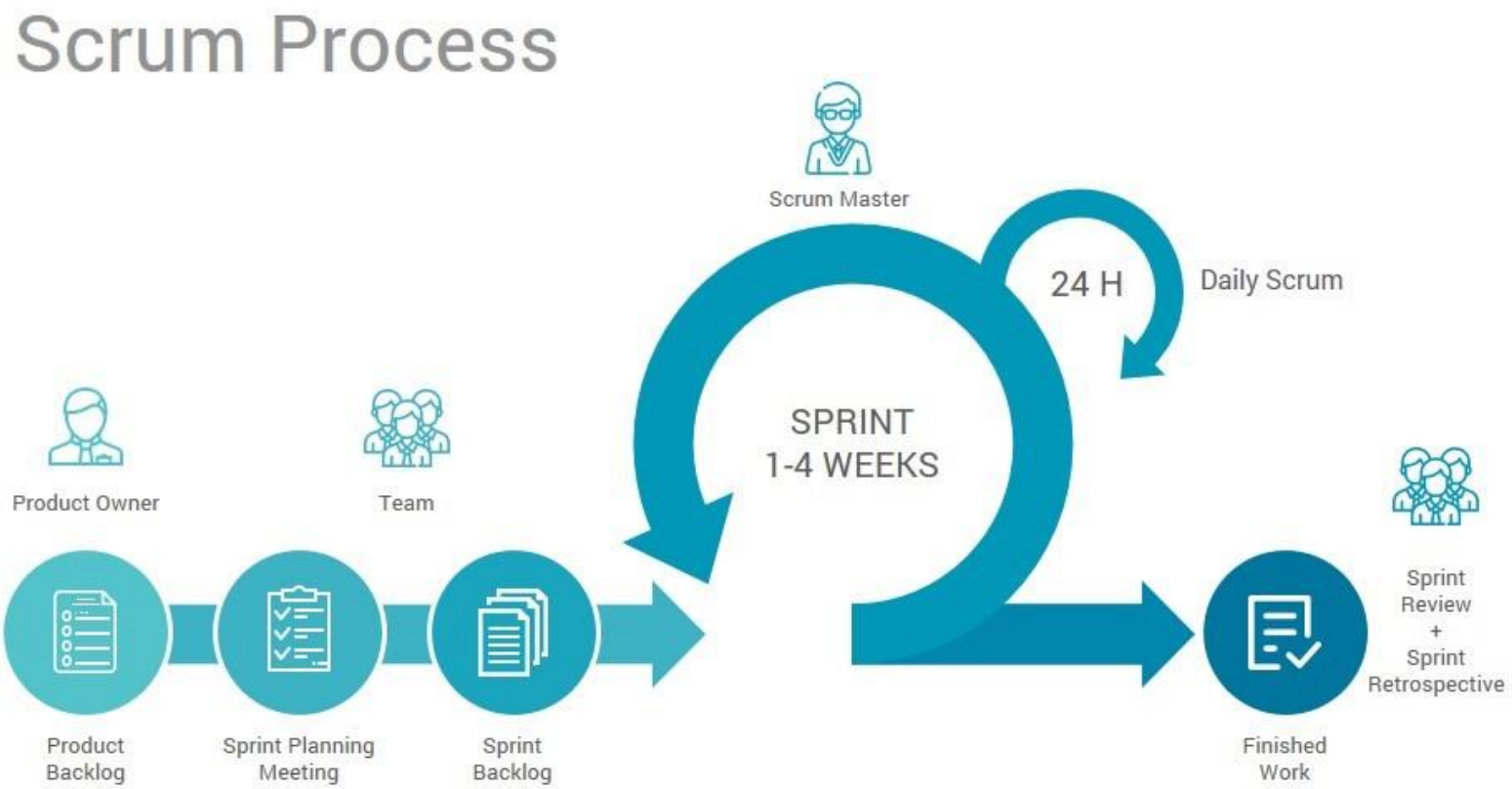
A spike is an experiment that allows developers to learn just enough about the unknown elements in a user story, e.g. a new technology, to be able to estimate that user story.

Often, a spike is a quick and dirty implementation, a prototype which will be thrown away. So, when a user story on the product backlog contains unknown elements that seriously hamper a usable estimation, the item should be split into a spike to investigate these elements plus a user story to develop the functionality.

Fuente: Erik Philippus, *Architecture Spikes*, july 2009.

metodología ágil

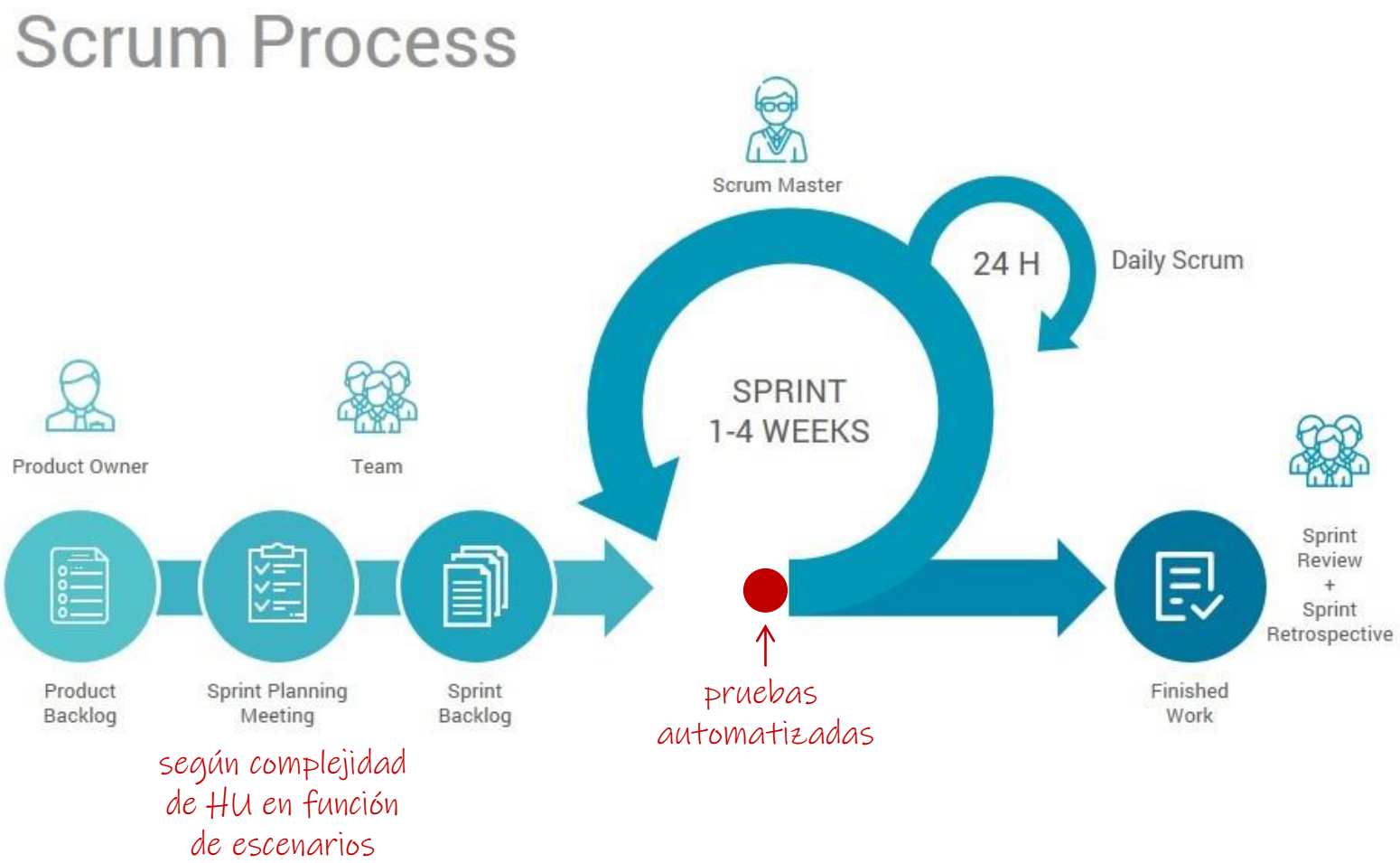
Scrum



Fuente: Allan Porras, *Scrum Methodology for Digital Product Development*, 4Geeks, nov 2019.

metodología ágil

Scrum



metodología ágil

miniiteración

Tras implementar las pruebas de aceptación vinculadas a las historias asignadas a la siguiente iteración (sprint), y tras comprobar que todas fallan, ATDD…

1. Identifica una prueba de aceptación objetivo.
2. Ejecuta todas las pruebas escritas hasta ese momento.
3. Escribe únicamente el código que permita pasar la prueba objetivo (long skinny slice).
4. Ejecuta *otra vez* todas las pruebas escritas hasta ese momento.
5. Refactoriza el código añadido.
6. Ejecuta *otra vez* todas las pruebas escritas hasta ese momento.
7. Vuelve al paso 1.

historias de usuario

criterios INVEST

Criterios **INVEST** (aplicables a historias de usuario)

Independientes: las historias no dependen entre sí.

Negociables: una historia de usuario no es un contrato.

Valiosas para el cliente: cada historia satisface una necesidad del cliente.

Estimables: suficiente detalle para permitir estimación de esfuerzo.

Small (pequeñas, simples): simples de implementar y de planificar.

Testable (validable): los criterios de validación son claros y explícitos.

trabajo en equipo

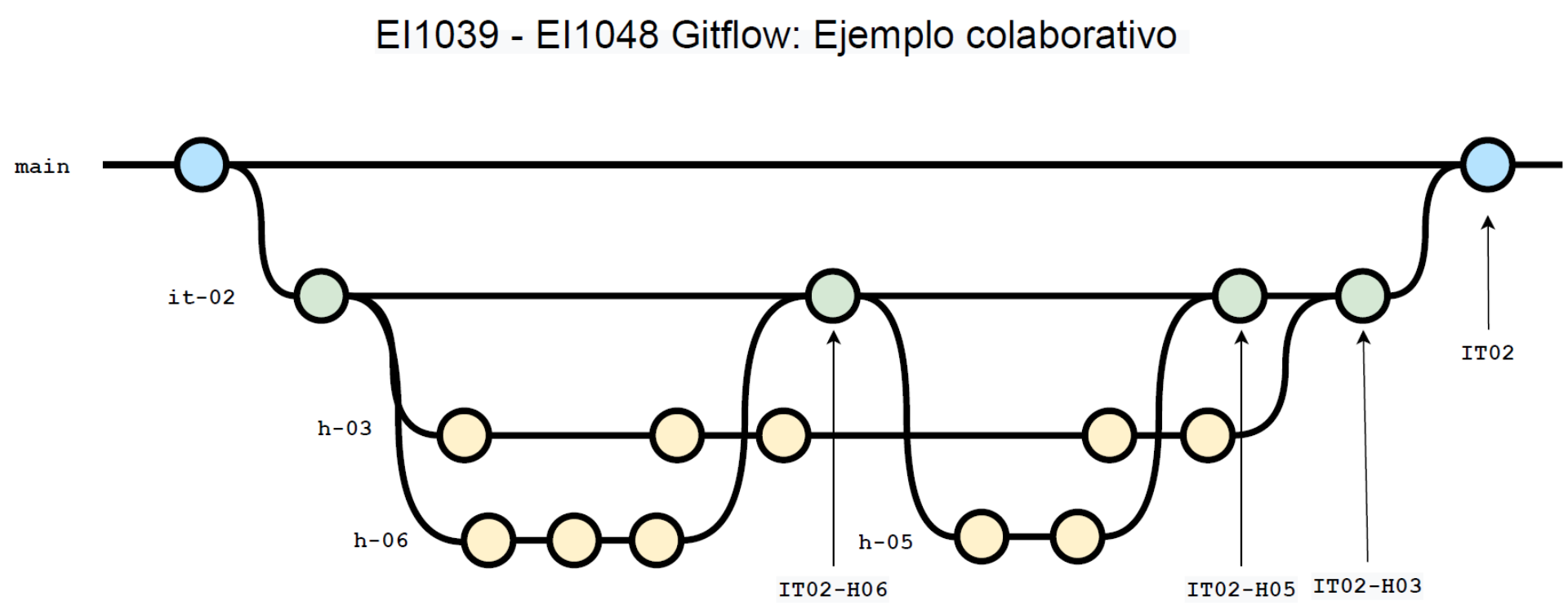
buenas prácticas

buenas prácticas…

- trabajo conjunto, recomendado para definición de escenarios, pruebas, elección de la arquitectura, planificación (estimación de historias y distribución en sprints)
- trabajo independiente, recomendado para prototipos de investigación de tecnologías (spike), implementación de pruebas, desarrollo de entidades (clases, métodos), etc.
- repositorio centralizado en la nube (p. ej., GitHub)
- trabajo independiente en entornos locales + integración frecuente en repo. común
- garantizar estabilidad del repositorio central, antes de integrar (subir) cambios:
  - actualizar repositorio local
  - construcción local
  - ejecución de todas las pruebas y validación de la consistencia de resultados
  - integra tus cambios en el repositorio centralizado

trabajo en equipo

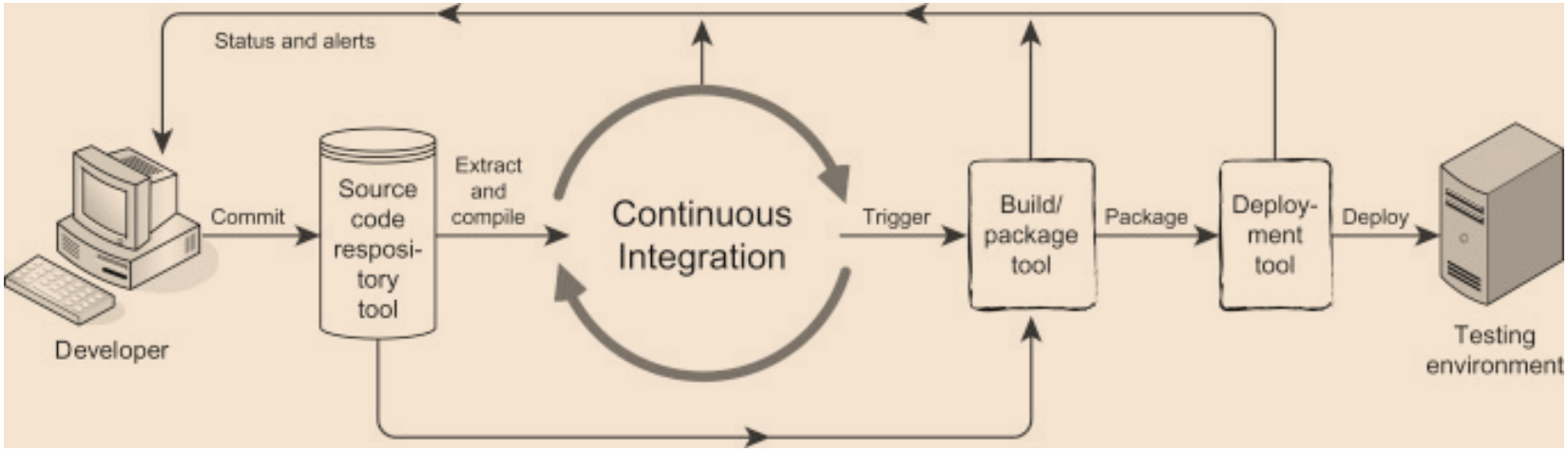
buenas prácticas



trabajo en equipo

buenas prácticas

integración continua (p. ej., GitHub Actions)



Fuente: Murat Erder, Pierre Pureur. *Continuous Architecture and Continuous Delivery*. Continuous Architecture, Morgan Kaufmann, pp 103-129, 2016.

recursos y herramientas

API públicas: sugerencias

Servicios públicos probados, sugeridos pero no obligatorios:

- openrouteservice: datos de calidad del aire y clima para una ubicación.
- datos.gob.es: datos de calidad de aire y clima para una ubicación.
- Geocode.xyz: datos de coordenadas a partir de un topónimo.
- API de Red Eléctrica de España: precios del kW/h.

entregas

1/3

Entrega 1: breve informe con los siguientes apartados…

- Requisitos, historias de usuario, escenarios, y pruebas de aceptación. Clasificación de escenario según tipo (válido o inválido) y justificación de necesidad. Cada prueba según esquema Given – When – Then.
- Código funcional de la iteración 0 (spike). Se proporcionarán las urls de acceso al repositorio del miniproyecto del spike (garantizar acceso al profesor de prácticas).
- Diseño de interfaces gráficas de usuario, por ejemplo, mock-ups que permitan entender cómo se cubrirá cada HU. Estos diseños deberán ser consistentes con la descomposición de HU en escenarios.
- Documentación de experiencias con IA generativa: tarea afectada, secuencia de interacciones, análisis respuestas, recomendaciones.

Fecha límite: viernes 16 de octubre

entregas

2/3

Entrega 2: breve informe con los siguientes apartados…

- Planificación inicial del desarrollo (distr. razonada de historias en iteraciones)
- Código de las pruebas de aceptación correspondientes a las historias de la iter. 1.
- Diagrama de clases UML necesario para ejecutar estas pruebas. ¿Ves la relación?
- URL de acceso al repositorio del proyecto principal (si es privado, recordad garantizar acceso a profesores). Incluirá el código ejecutable de las pruebas, junto al de las clases y métodos necesarios para la ejecutar las pruebas (para fallar).
- Análisis y justificación del patrón de diseño seleccionado para la interfaz de usuario (MVC, MVVC, etc).
- Análisis preliminar de los patrones de diseño (al menos 5) potencialmente útiles.
- Documentación de experiencias con IA generativa: tarea afectada, secuencia de interacciones, análisis respuestas, recomendaciones.

Fecha límite: viernes 13 de noviembre

entregas

3/3

Entrega 3: informe final con los siguientes apartados…

- Resultados por iteraciones: historias terminadas, pruebas implementadas, resultados de las pruebas, impacto de los resultados en la planificación inicial.
- Listado de pruebas de aceptación implementadas, justificación y explicación.
- Listado de pruebas de integración implementadas, justificación, y explicación. Detalles sobre la necesidad y configuración de los mocks utilizados.
- Implementación de patrones de diseño, justificando su idoneidad para el proyecto.
- Implementación del patrón de diseño utilizado en las GUI, justificando usabilidad.
- Análisis crítico del diseño de la GUI presentado en la Entrega 1. Incluir interfaces mock-ups e interfaces definitivas para valorar la evolución entre entregas.
- Conclusiones, grado de cumplimiento de objetivos y experiencia personal.

Fecha límite: enero de 2027

entregas

3/3

Si alguna entrega no es aceptable, el profesorado la marcará con una tarjeta amarilla, es decir, un aviso para que el equipo correspondiente mejore considerablemente su proyecto para la próximo entrega. Si el nivel en la siguiente entrega sigue siendo deficiente, el profesorado la marcará con una tarjeta roja, lo cual implica un suspenso automático del proyecto para el equipo en la convocatoria en vigor.

entregas

3/3

la corrección de la entrega final incluirá…

- construcción del proyecto
- ejecución de todas las pruebas implementadas
- ejecución de la aplicación para evaluar cumplimiento de requisitos funcionales
- defensa de las decisiones de diseño
- evaluación del informe.

recordatorio notas máximas

- proyecto básico, nota ≤ 8
- proyecto avanzado, nota ≤ 10

tutoría obligatoria (enero de 2024)

- defensa única del proyecto para ambas asignaturas

criterios de evaluación

SI puede acreditarse desarrollo según ATDD, la app funciona, se implementa número mínimo de patrones de diseño y no se ha recibido tarjeta roja…

- calidad del producto final, considerando alcance funcional, tecnologías usadas, usabilidad, estética, etc. [2 puntos]
- historias, escenarios, pruebas de aceptación e integración, implementación de las pruebas, uso y diseño de mocks, etc. [3 puntos]
- patrones de diseño, idoneidad de las soluciones de diseño propuestas, grado de cumplimiento del objetivo de arquitectura desacoplada, etc. [3 puntos]
- calidad de los informes entregados y de la presentación, considerando estructura, estilo de redacción, uso del vocabulario técnico, ortografía, precisión y valor de los argumentos, entregas en plazo, etc. [2 puntos]

SINO

- nota de proyecto = 0

uso de IA generativa

Se fomentará el uso de la IA generativa, sujeto a las siguientes condiciones:

- uso transparente (deberá informarse y documentarse)
- uso honesto (se documentarán todas las experiencias de uso)
- uso responsable (no se abusará de su uso)
- documentación: ámbito, secuencia de interacciones (contextos + preguntas)
- análisis crítico de respuestas: valor para el proyecto, grado de creatividad, beneficios, limitaciones, recomendaciones
- se penalizarán evidencias de uso no documentadas, o usos acríticos de respuestas automáticas que conduzcan a soluciones inadecuadas, ineficientes o incorrectas.